P0784R2, 2018-05-05
EWG, LEWG

Louis Dionne (ldionne.2@gmail.com)
Richard Smith (richard@metafoo.co.uk)
Nina Ranns (dinka.ranns@gmail.com)
Daveed Vandevoorde (daveed@edg.com)

Standard containers and constexpr

R0: Original proposal, presented in Albuquerque 2017.
EWG approved the direction: SF: 11 | F: 12 | N: 2 | A: 0 | SA: 0
R1: Added initial wording.
R2: Implemented core review comments
P0784R2: removed special treatement of static storage duration destructors

Introduction and motivation

Variable size container types, like `std::vector` or `std::unordered_map`, are generally useful for runtime programming, and therefore also potentially useful in constexpr computations. This has been made clear by some recent experiments such as the Constexpr ALL the things! presentation (and its companion paper P0810R0 to be published in the pre-Albuquerque mailing) by Ben Deane and Jason Turner, in which they build a compile-time JSON parser and JSON value representation using `constexpr`. Amongst other things, the lack of variable size containers forces them to use primitive fixed-size data structures in the implementation, and to parse the input JSON string twice; once to determine the size of the data structures, and once to parse the JSON into those structures.

We also expect variable size containers to be a necessity in the reflection and metaprogramming APIs that will emerge from the work in SG-7, which decided that the preferred direction for a standard solution would involve constexpr-like computation. For example, querying the template arguments of a class type might look something like:

```
std::vector<std::metainfo> args = std::meta::get_template_args(reflexpr(T));
```

There are three aspects of `std::vector` that make it currently unusable in constexpr evaluations:

1. Destructors cannot be constexpr.
2. Dynamic memory allocation/deallocation isn't available.
3. In-place construction using placement-new isn't available.

Destructors

The limitation that destructors cannot be constexpr is somewhat artificial: We can just lift the restriction. We have discussed the issues with the implementers of MSVC++ (Microsoft), GCC, Clang, and EDG's front end, and they all agreed that it would entail at most a minor cost in the performance of constexpr evaluations.

The proposed rules for constexpr destructors are:

- Destructors can be declared constexpr.
- Defaulted destructors are implicitly constexpr if their implementation does not call non-constexpr destructors.

- As an important special case, trivial destructors are implicitly constexpr.
- A literal type requires a constexpr destructor (previously, the stronger requirement of a trivial destructor was made).
- An object that has been destroyed (and not reconstructed --- see later) cannot be accessed during constexpr evaluation, even if its destructor is trivial.

For a constexpr variable to be declared, we would extend the requirement that the variable has a constant initializer with a second requirement that the variable has *constant destruction*. This means that the evaluation of the variable's destructor on the constant value produced by the initializer must also be a constant expression.

Memory allocation

The memory implementation of the constexpr evaluator is unlike that of typical normal (run-time) program evaluation.Ã‚Â For example, it *must* be able to catch any form of undefined behavior. That means that the representation of a pointer or reference cannot just be an address: Additional metadata is needed to be able to relate the pointer to the bounds of the object it points into. Another example: Metadata is needed to know which field of a union is active.

Because of this, casting a raw memory pointer (say, a `void*`) into a pointer to an actual object is not generally viable. That removes the option of using something like

```
void* operator new(std::size_t);
```

during constexpr evaluation: There is no reasonable way to turn the `void*` back into a `T*`.

Instead we can contemplate two other options:

1. Deal in terms of (non-placement) new- and delete-expressions.
2. Deal in terms of the standard allocator.

Both of these provide *typed* storage that doesn't require further reinterpretation. We can therefore establish rules that make them work "magically" during constexpr evaluation (without evaluating the underlying raw storage pointer arithmetic or pointer reinterpretations). We'll discuss those rules further on, but one guiding principle is that memory that is dynamically allocated during constexpr evaluation cannot just "escape" into the run time implementation.

In-place construction

Standard containers rely on the ability to separate the allocation and construction of objects through the `std::allocator_traits` interface. In particular, `std::allocator_traits<A>::construct` is a typed API that just turns around and calls a non-typed placement-new-expression ("non-typed" because the extra-parameter of the placement-new operator is `void*`). Although we cannot make the general placement-new mechanism work in constexpr evaluation, we *can* decree that:

```
new(ptr) T{...}
```

is a valid core constant expression if `ptr` is obtained by a standard conversion from a `T*` pointer value that points to a "dead object":

- the storage of an object of type `T` that has been destroyed and not yet (in part or in whole) reconstructed, or
- type `T` storage that has been obtained from the default allocator and in which no object has yet been constructed.

Requiring that the storage being constructed into is "dead" makes for a clean semantic model. However, for non-class types, pseudo-destructor calls don't currently end the lifetime of the underlying object. We are therefore left

with three options:

- permit construction over a "live" object (preserving its complete type), or
- add general semantics to pseudo-destructor calls ending the lifetime of the underlying objects, or
- add semantics to pseudo-destructor calls ending the lifetime of the underlying objects during constexpr evaluation only.

For example:

```
#include <memory>
#include <new>

constexpr int f() {
  std::allocator<double> a;
  double *b = a.alloc(1);
  new (b) double{3.3};

  // Does constexpr evaluation fail because `b` already
  // points to a live object?
  new (b) double{4.4};

  a.deallocate(b, 1);
  return 0;
}

constexpr int evaluate_as_constexpr = f();
```

Non-transient allocation

During a constexpr evaluation, any allocated storage that is deallocated before the evaluation completes poses few problems: We call those constexpr allocations *transient*. We'll decree that the overall result of a constant expression cannot contain a pointer or reference to storage from a transient allocation. (Note that since C++14, a compiler is allowed to optimize away certain allocations and deallocations, and the *transient* constexpr allocation rules can be interpreted as mandating such elision for constexpr evaluations.)

What about storage that hasn't been deallocated by the time evaluation completes? We could just disallow that, but there are really compelling use cases where this might be desirable. E.g., this could be the basis for a more flexible kind of "string literal" class. We therefore propose that if a non-transient constexpr allocation is valid (to be described next), the allocated objects are promoted to static storage duration. A constexpr evaluation of an expression *expr* can refer to a *non-transient* allocation if:

- *expr* is a full expression in a context that requires a constant expression, and
- the result of evaluating *expr* is an object with a nontrivial constexpr destructor, and
- evaluating that destructor would be a valid core constant expression and would deallocate all the *non-transient* allocations produced by the evaluation of *expr*.

For example:

```
#include <memory>
#include <new>
using namespace std;
template<typename T> struct S: allocator<T> {
  T *ps;
  int sz;
  template<int N> constexpr S(T (&p)[N])
                          : sz{N}
                          , ps{this->allocate(N)} {
    for (int k = 0; k<N; ++k) {
      new(this->ps+k) T{p[k]};
```

```
    }
  }
  constexpr ~S() {
    for (int k = 0; k<this->sz; ++k) {
      (this->ps+k)->T::~T();
    }
    this->deallocate(this->ps, this->sz);
  }
};

constexpr S<char> str("Hello!");
  // str ends up pointing to a static array
  // containing the string "Hello!".
```

The constructor constexpr evaluation in this example is successful, producing an `S` object that points to a non-transient constexpr allocation. The constexpr evaluation of the destructor would also be successful and would deallocate the non-transient allocation. The non-transient allocation is therefore promoted to static storage.

Object lifetime issues

The current rules regarding object storage reuse (in [basic.life]) for objects that contain immutable data or objects that contain variant (i.e., union) members are either subtle or in need of revision. We think it is acceptable for constexpr evaluation to put stronger constraints on this than the general abstract machine, but the reverse is probably not acceptable. For now, we therefore propose to disallow placement new for types that contain const subobjects, references, or union subobjects (including anonymous unions). For example:

```
#include <new>
struct S {
  int const ic;
};
constexpr int f() {
  S s{41};
  s.~S();
  new (&s) S{42};  // Not a core constant expression.
  return s.ic;
}
constexpr int r = f();  // Error.
```

We're hopeful, however, that we will be able to lift that restriction when the general object model has been cleaned up.

Library pragmatism

Current implementations of standard libraries sometimes perform various raw storage operations through interfaces other than the standard allocator and allocator traits. That may make it difficult to make the associated components usable in constexpr components. Based on a cursory examination of current practices, we therefore propose to start only with the requirement that the container templates in the [containers] clause be usable in constexpr evaluation, when instantiated over literal types and the default allocator. In particular, this excludes `std::string`, `std::variant`, and various other allocating components. Again, it is our hope we will be able to extend support to more components in the future.

With regards to the default allocator and allocator traits implementation, the majority of the work is envisioned in the constexpr evaluator: It will recognize those specific components and implement their members directly (without necessarily regarding the library definition).
We might, however, consider decorating the class members with the `constexpr` keyword. Also, some implementations provide extra members in these class templates (such as libc++'s `allocator_traits<A>::__construct_forward`) that perform non-constexpr-friendly operations (`memcpy`, in

particular). Lifting such members to standard status would help interoperability between library and compiler implementations.

Implementation experience

So far, this has not been implemented. However, based on preliminary discussion with implementers working on Clang, MSVC and EDG, no blockers that would make this feature unimplementable or prohibitively expensive to implement have been identified at the moment.

Wording changes

Note: *The following changes enable "constexpr destructors". See further down for allocation-related changes.*

Change in [basic.types] paragraph 10.5.1:

> -it has a ~~trivial~~**constexpr** destructor,

Change in [expr.const] paragraph 2:

> An expression `e` is a *core constant expression* unless the evaluation of `e`, following the rules of the abstract machine (4.6), would evaluate one of the following expressions:
>
> — `this` (8.1.2), except in a constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor** that is being evaluated as part of e;
>
> — an invocation of a function other than a constexpr constructor **or destructor** for a literal class,**or** a constexpr function ~~, or an implicit invocation of a trivial destructor (15.4)~~ [*Note:* Overload resolution (16.3) is applied as usual — *end note*];
>
> — an invocation of an undefined constexpr function ~~or~~**,** an undefined constexpr constructor, **or an undefined constexpr destructor**;
>
> — an invocation of an instantiated constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor** that fails to satisfy the requirements for a constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor** (10.1.5);
>
> — an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either
>
> — it is initialized with a constant expression**,** ~~or~~
>
> — its lifetime began within the evaluation of `e`**, or**~~;~~
>
> **— for an *id-expression* that refers to a data member of a constexpr variable `x` with static storage duration, the evaluation occurs during the invocation of a constexpr destructor for `x`.**
>
> — modification of an object (8.18, 8.2.6, 8.3.2) unless it is applied to a non-volatile lvalue of literal type that refers to ~~non-volatile object whose lifetime began within the evaluation of `e`~~
>
> **— a non-volatile object whose lifetime began within the evaluation of `e`, or**
>
> **— a non-volatile subobject of a constexpr variable `x` with static storage duration and the modification occurs in an invocation of a constexpr destructor for `x`**

Add new paragraph after [expr.const] paragraph 2:

> **An object is said to have *constant destruction* if:**
>
> > **— it is of a non-class type, or**
> >
> > **— the type of that object has a constexpr destructor and for a hypothetical expression e whose only effect is to destroy that object, e is a core constant expression.**

Change in [dcl.constexpr] paragraph 2:

> A `constexpr` specifier used in the declaration of a function that is not a constructor **or a destructor** declares that function to be a *constexpr function*. Similarly, a `constexpr` specifier used in a constructor declaration declares that constructor to be a *constexpr constructor*. **A `constexpr` specifier used in a destructor declaration declares that destructor to be a *constexpr destructor*.**

Insert new paragraph after [dcl.constexpr] paragraph 4:

> **The definition of a `constexpr` destructor shall satisfy the following requirements:**
>
> > **— it shall not be virtual**
> >
> > **— the class shall not have any virtual base classes;**
> >
> > **— its *function-body* shall satisfy the requirements for a *function-body* of a constexpr function;**

Change in [dcl.constexpr] paragraph 6:

> If the instantiated template specialization of a constexpr function template or member function of a class template would fail to satisfy the requirements for a constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor**, that specialization is still a constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor**, even though a call to such a function cannot appear in a constant expression. If no specialization of the template would satisfy the requirements for a constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor** when considered as a non-template function ~~or~~**,** a constructor, **or a destructor**, the template is ill-formed, no diagnostic required.

Change in [dcl.constexpr] paragraph 8:

> The constexpr specifier has no effect on the type of a constexpr function ~~or~~**,** a constexpr constructor, **or a constexpr destructor**.

Change in [dcl.constexpr] paragraph 9:

> In any `constexpr` variable declaration, the full-expression of the initialization shall be a constant expression (8.20). **A `constexpr` variable shall have constant destruction.**

Change in [class.dtor] paragraph 1:

> Each decl-specifier of the decl-specifier-seq of a destructor declaration (if any) shall be `friend`, `inline`, ~~or~~ `virtual`**, or `constexpr`**.

Add after [class.dtor] paragraph 9:

> **The defaulted destructor is a constexpr destructor if**

— it is a trivial destructor, or

— it is not virtual and all the destructors it invokes are constexpr destructors

Note: *The following changes enable some "constexpr new-expressions".*

Modify [expr.new] paragraph 10

An implementation is allowed to omit a call to a replaceable global allocation function (21.6.2.1, 21.6.2.2). When it does so, the storage is instead provided by the implementation or provided by extending the allocation of another *new-expression.*

~~The implementation may extend the allocation of a new-expression `e1` to provide storage for a *new-expression* `e2` if the following would be true were the allocation not extended:~~

~~— the evaluation of `e1` is sequenced before the evaluation of `e2`, and~~

~~— `e2` is evaluated whenever `e1` obtains storage, and~~

~~— both `e1` and `e2` invoke the same replaceable global allocation function, and~~

~~— if the allocation function invoked by `e1` and `e2` is throwing, any exceptions thrown in the evaluation of either `e1` or `e2` would be first caught in the same handler, and~~

~~— the pointer values produced by `e1` and `e2` are operands to evaluated delete-expressions, and~~

~~— the evaluation of `e2` is sequenced before the evaluation of the delete-expression whose operand is the pointer value produced by `e1`.~~

~~[*Example:*~~

~~...~~

~~— *end example*]~~

Add new paragraph after [expr.new] paragraph 10

During an evaluation of a constant expression, a call to an allocation function is always omitted. [Note: Only *new-expression*s that would otherwise result in a call to a replaceable global allocation function can be evaluated in constant expressions (see [expr.const]). — *end note*]

Add new paragraph after [expr.new] paragraph 10

The implementation may extend the allocation of a *new-expression* `e1` to provide storage for a *new-expression.* `e2` if the following would be true were the allocation not extended:

— the evaluation of `e1` is sequenced before the evaluation of `e2`, and

— `e2` is evaluated whenever `e1` obtains storage, and

— both `e1` and `e2` invoke the same replaceable global allocation function, and

— if the allocation function invoked by `e1` and `e2` is throwing, any exceptions thrown in the evaluation of either `e1` or `e2` would be first caught in the same handler, and

— the pointer values produced by `e1` and `e2` are operands to evaluated *delete-expressions*, and

— the evaluation of `e2` is sequenced before the evaluation of the *delete-expression* whose operand is the pointer value produced by `e1`.

[*Example:*

...

— *end example*]

Change in [expr.const] paragraph 2:

~~— a *new-expression (8.3.4);*~~

— a *new-expression* (8.3.4), unless the selected allocation funcstion is a replaceable global allocation function (21.6.2.1, 21.6.2.2);

~~— a *delete-expression (8.3.5);*~~

Add to end of [expr.const] paragraph 6:

Every object of dynamic storage duration [6.6.4.4] created during the evaluation of a constant expression shall be destroyed during that evaluation and its storage shall be deallocated.

Note: *The following changes enable the use of the default allocator in constant expressions.*

Add a new paragraph after [expr.const] paragraph 2:

For the purposes of determining whether an expression is a core constant expression, the evaluation of a call to a member function of `std::allocator<T>` as defined in _allocator.members_, where `T` is a literal type, is permitted even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression. Similarly, the evaluation of a call to a member function of `std::allocator_traits<std::allocator<T>>` as defined in _allocator.traits,members_, is a valid core constant expression unless

— for a call to the member `construct`, the evaluation of the underlying constructor call is not a core constant expression, or

— for a call to the member `destroy`, the evaluation of the underlying destructor call is not a core constant expression.